REPL, SDK, 확장과 에이전트 기능

하나의 실행 엔진이 터미널 UI·headless SDK·MCP·plugin·skill·subagent라는 서로 다른 표면으로 투영되는 방식을 읽는다

이 권을 마치면 UI와 generator를 프로젝트 전체로 과대평가하지 않으면서도, core runtime을 사람이 쓰고 외부 시스템이 확장하게 만드는 결정적 adapter로 설명할 수 있다.

STUDY SNAPSHOT
Claude Code v2.1.88
2ca5ddabfed5

제3자 복원 소스 기반
한국어 · Guided Edition

오늘 우리는 이 코드의 어느 부분을 함께 보는가

사용자는 REPL을 통해 시스템을 경험하지만 같은 core는 SDK와 remote session에서도 소비된다. MCP와 plugin은 외부 capability를 들이고, skill은 instruction을 들이며, AgentTool과 task runtime은 실행을 다시 중첩한다. 이 권은 이들을 하나의 "부가기능" 목록으로 묶지 않고 core와 만나는 계약을 따라간다.

694
분류된 TS/TSX 파일

164,743
물리 코드 줄

476
async 함수

966
tool use 표기

CHAPTER 01

REPL은 canonical state의 대화형 projection이다

Ink component tree는 terminal을 그리지만, 실행의 진실을 소유하기보다 engine state와 event를 사용자가 조작 가능한 화면으로 바꾼다.

CHAPTER 02

SDK와 remote는 같은 엔진의 다른 소비자다

화면을 제거한 것이 SDK가 아니다. schema, control message, backpressure, permission callback으로 core protocol을 외부에 안정적으로 노출해야 한다.

CHAPTER 03

MCP, plugin, skill은 서로 다른 것을 확장한다

MCP는 외부 도구/자원 protocol, plugin은 설치·배포 단위, skill은 model instruction package다. 같은 확장 목록으로 취급하면 trust 경계가 흐려진다.

CHAPTER 04

Agent와 Task는 실행을 한 단계 더 중첩한다

subagent는 함수 호출이 아니라 독립 context·budget·permission·result projection을 가진 child runtime이며, task는 그 수명과 관찰을 관리한다.

분포 수치는 중요도 점수가 아니다. 이 권이 직접 펼쳐보는 코드 표면의 규모와 언어 메커니즘 밀도를 가늠하기 위한 탐색 표지다.

01

REPL은 canonical state의 대화형 projection이다

Ink component tree는 terminal을 그리지만, 실행의 진실을 소유하기보다 engine state와 event를 사용자가 조작 가능한 화면으로 바꾼다.

이 장에서 실제로 펼쳐볼 경로

src/screens/REPL.tsx · src/components/PromptInput/** · src/ink/** · src/context/**

REPL.tsx의 크기는 화면 수가 아니라 조정 책임의 수다

메시지 rendering, 입력, modal, permission, background task, compact가 한 terminal session에서 만난다.

REPL.tsx는 단순 layout component가 아니다. engine event를 state에 반영하고, 여러 context provider와 overlay를 결합하며, 키 입력이 실행 제어로 이어지게 한다.

React/Ink를 쓴다고 상태가 자동으로 정리되는 것은 아니다. durable session, engine in-flight state, ephemeral UI state가 한 component에 섞이면 rerender와 resume가 서로 영향을 준다.

분해할 때 화면 영역별로 자르기 전에 state owner를 나누자. transcript projection, current turn controller, overlay navigation은 변경 이유와 수명이 다르다.

스터디에서 확인할 질문

REPL state 중 process crash 뒤 복구되어야 할 것과 즉시 버려도 되는 것을 각각 다섯 개 골라 보라.

React render는 effect execution과 분리되어야 한다

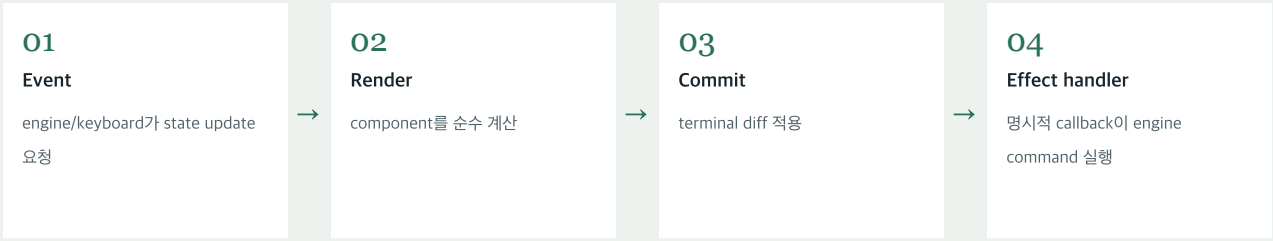
component 함수는 화면 계산이고, tool 실행이나 session append는 명시적 event/effect 경계에서 일어나야 한다.

React의 render/commit 구분은 terminal에서도 유효하다. 같은 props와 state로 component가 다시 호출될 수 있으므로 render 중 외부 효과를 수행하면 중복 실행 위험이 생긴다.

Ink는 DOM 대신 terminal output을 commit하지만 declarative component model은 같다. 화면을 다시 계산하는 과정과 process에 signal을 보내는 과정은 별도 callback/effect로 유지해야 한다.

UI 버그를 조사할 때 rerender 횟수만 세지 말고 어떤 event가 state를 바꾸고 어떤 effect가 그 event를 발생시켰는지 역추적하자. 그래야 core와 UI 사이 feedback loop를 찾을 수 있다.

Ink 화면 갱신과 외부 effect의 경계



이 그림이 감추지 않는 것 React는 component를 다시 호출할 수 있다. render가 pure해야 동일 입력에서 중복 process/tool effect가 발생하지 않는다.

스터디에서 확인할 질문

Strict한 반복 render를 가정했을 때 중복 실행되면 안 되는 callback이 component body 안에 숨어 있지 않은가?

함께 펼쳐볼 자료 src/screens/REPL.tsx · src/ink.ts · React render and commit · Ink repository and component model

PromptInput은 텍스트 박스이면서 mode-aware controller다

history, multiline, paste, voice, queued command, suggestion이 입력의 의미를 바꾼다.

PromptInput 디렉터리는 main component뿐 아니라 input mode, paste 처리, footer suggestion, history search를 분리한다. 이는 입력 UX가 이미 작은 subsystem임을 보여 준다.

사용자가 보는 문자열과 submit되는 content block이 다를 수 있으므로, caret와 display state를 canonical prompt로 오해하면 안 된다. 특히 image paste나 command completion은 typed input을 만든다.

새 입력 기능은 keystroke handler에 바로 넣기보다 input state machine의 transition으로 보자. normal/history-search/permission/modal 사이 keybinding 충돌을 체계적으로 검토할 수 있다.

스터디에서 확인할 질문

escape와 enter가 mode마다 다른 의미를 가질 때 keybinding resolution의 우선순위를 어디에서 소유해야 하는가?

Context provider는 dependency injection이면서 rerender boundary다

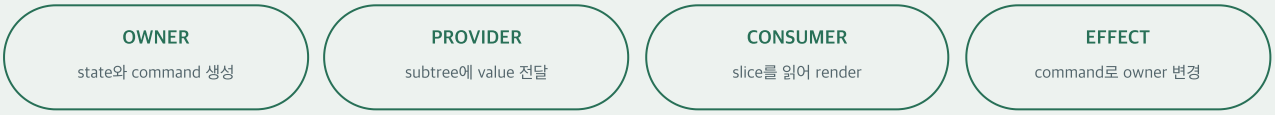
modal, mailbox, notifications, queued message가 전역 singleton 대신 tree 범위 상태를 제공한다.

React context는 깊은 component에 state와 command를 전달한다. 이 덕분에 prop drilling은 줄지만 provider value가 넓으면 작은 변경이 큰 tree를 다시 render하고 ownership이 흐려질 수 있다.

context 이름만 보고 domain boundary라고 단정하면 안 된다. 일부는 순수 UI 상태이고 일부는 engine event를 반영한다. provider가 canonical state를 복제하는지, projection만 저장하는지 확인해야 한다.

개선할 때 context 수를 줄이는 것이 목표가 아니다. read model과 command port를 분리하고, consumer가 필요한 최소 slice만 구독하게 만드는 것이 더 직접적이다.

Provider value가 만드는 두 가지 결합



`value identity change` consumer rerender

`command()` owner state transition

`wide value` 불필요한 consumer까지 갱신

`duplicated state` canonical/projection drift

언어·런타임 안쪽에서는 Context는 전역 변수 대체재이지만 domain boundary를 자동 생성하지 않는다. read model과 command port를 나눠야 ownership이 선명해진다.

스터디에서 확인할 질문

QueuedMessageContext의 state는 engine queue와 같은 진실인가, UI가 보여 주기 위한 projection인가?

함께 펼쳐볼 자료 src/context/QueuedMessageContext.tsx · src/context/modalContext.tsx · src/context/notifications.tsx

02

SDK와 remote는 같은 엔진의 다른 소비자다

화면을 제거한 것이 SDK가 아니다. schema, control message, backpressure, permission callback으로 core protocol을 외부에 안정적으로 노출해야 한다.

이 장에서 실제로 펼쳐볼 경로

src/QueryEngine.ts · src/entrypoints/sdk/** · src/bridge/** · src/remote/**

SDK message는 내부 event의 공개 projection이다

외부 caller에게 필요한 정보와 내부 구현 세부를 구분해야 버전 가능한 계약이 된다.

QueryEngine은 internal message와 state를 SDKMessage로 변환해 yield한다. permission denial, usage, partial message 포함 여부처럼 headless consumer에게 필요한 사건이 명시된다.

내부 object를 그대로 노출하면 빠르게 만들 수 있지만 필드 rename 하나가 public break가 된다. 반대로 지나치게 단순한 result만 주면 host가 progress와 control을 구현할 수 없다.

SDK contract를 바꿀 때는 additive field, new union variant, changed ordering을 구분하자. 소비자가 exhaustive switch를 쓸 수 있으므로 새 variant도 호환성 변화다.

Internal event에서 public SDK contract로



이 그림이 감추지 않는 것 projection은 복사가 아니라 안정성 경계다. 내부 field를 그대로 노출하면 리팩터링이 public breaking change가 된다.

스터디에서 확인할 질문

내부에는 있지만 SDK에 내보내지 말아야 할 정보의 기준을 security, stability, usefulness로 정리해 보라.

control schema는 host와 engine 사이의 양방향 protocol이다

질문을 보내고 답을 받는 SDK는 단순 event stream보다 복잡하다.

permission prompt, interrupt, set model 같은 control은 host가 engine 상태를 바꾸는 메시지다. runtime validation schema가 있어야 언어 경계를 넘은 잘못된 message를 초기에 거부할 수 있다.

이 protocol에는 correlation id와 timeout이 필요하다. permission request에 늦게 도착한 응답이 다음 요청에 적용되면 authority가 잘못 연결된다.

control을 추가할 때 request/response 형태뿐 아니라 idempotency, late response, disconnect 중 상태를 설계하자. 이것이 SDK를 안정적 transport contract로 만든다.

스터디에서 확인할 질문

host가 disconnect된 동안 열린 permission request는 auto-deny, pause, cancel 중 어떤 상태로 수렴해야 하는가?

generator 소비 속도가 backpressure를 만든다

SDK caller가 event를 읽지 않으면 producer의 진행과 buffer 정책이 영향을 받는다.

async iterator는 consumer가 다음 값을 요청하는 구조라 자연스러운 backpressure를 제공할 수 있다. 그러나 내부 callback이 별도 queue에 계속 push하면 queue가 무한히 자랄 수 있다.

UI는 빠르게 render할 수 있지만 network bridge는 느리거나 끊길 수 있다. 같은 event source라도 drop 가능한 progress와 반드시 전달할 terminal event를 구분해야 한다.

bridge를 설계할 때 sequence number와 resume cursor를 고려하면 disconnect 후 projection을 다시 맞출 수 있다. 다만 실제 tool effect의 exactly-once를 transport alone으로 보장할 수는 없다.

스터디에서 확인할 질문

progress event를 버려도 되는 최대 조건과 절대 버리면 안 되는 settlement event를 구분해 보라.

remote permission bridge는 authority를 네트워크 너머로 전달한다

원격 UI가 승인하더라도 요청 identity와 현재 tool input이 정확히 일치해야 한다.

remotePermissionBridge와 session manager는 permission 질문과 응답을 원격 transport에 연결한다. 지연, reconnect, 중복 delivery가 로컬 dialog에는 없던 실패를 만든다.

승인 token을 session-wide boolean로 만들면 재사용 공격과 잘못된 호출 승인이 가능하다. tool use id, normalized input digest, expiry와 결합해야 한다.

원격 제어를 추가할 때 local fallback을 명확히 하자. 연결이 끊겼다는 이유로 bypass하거나 무한 대기하면 각각 안전성과 availability가 깨진다.

스터디에서 확인할 질문

원격 승인 후 실행 직전 input이 hook에 의해 바뀌면 기존 승인을 무효화해야 하는가?

03

MCP, plugin, skill은 서로 다른 것을 확장한다

MCP는 외부 도구/자원 protocol, plugin은 설치·배포 단위, skill은 model instruction package다. 같은 확장 목록으로 취급하면 trust 경계가 흐려진다.

이 장에서 실제로 펼쳐볼 경로

src/services/mcp/** · src/services/plugins/** · src/plugins/** · src/skills/**

MCP adapter는 외부 capability를 Tool 언어로 번역한다

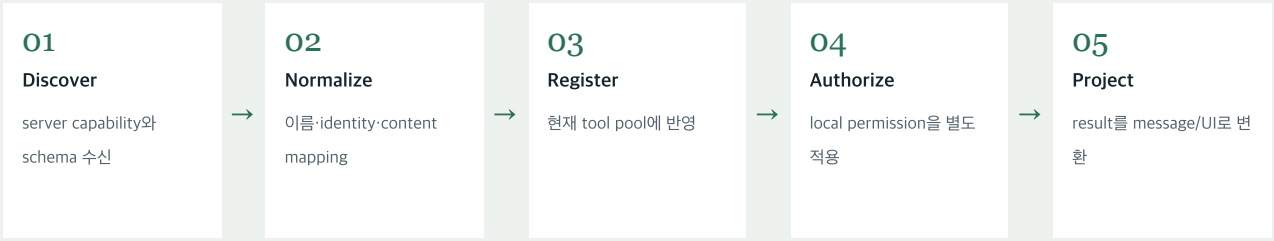
server가 광고한 schema와 transport event를 내부 registry와 permission model에 맞춰야 한다.

MCP client/normalization 코드는 server description, tool name, input schema, result content를 내부 형태로 바꾼다. 외부 server가 준 문자열은 신뢰된 코드가 아니므로 이름 충돌과 malformed schema를 다뤄야 한다.

MCP specification이 interoperability를 제공해도 로컬 authority는 따로다. server 연결 성공은 tool 실행 허가가 아니며, resource read와 tool effect의 위험도도 다르다.

새 MCP transport를 추가할 때 protocol mapping과 trust policy를 분리하자. stdio, HTTP, in-process는 인증과 프로세스 경계가 다르지만 내부 Tool contract는 같을 수 있다.

MCP server에서 내부 Tool까지



이 그림이 감추지 않는 것 protocol interoperability와 local trust는 별개다. 외부 schema는 runtime validation 입력이지 authority 증거가 아니다.

스터디에서 확인할 질문

서로 다른 MCP server가 같은 display name의 tool을 제공할 때 identity와 permission rule을 어떻게 안정적으로 만들 것인가?

함께 펼쳐볼 자료 src/services/mcp/client.ts · src/services/mcp/normalization.ts · src/services/mcp/types.ts · Model Context Protocol specification

MCP connection manager는 상태 기계다

connecting, connected, failed, reconnecting 상태와 tool catalog 변화가 함께 움직인다.

MCPConnectionManager와 관리 UI는 server lifecycle을 추적한다. connection이 끊기면 tools와 instructions가 사라지고, model context에도 delta가 전달되어야 한다.

연결 상태를 boolean 하나로 두면 재시도 중인지 영구 실패인지 구분할 수 없다. 또한 tool list를 즉시 제거할지 진행 중 호출을 마칠지 정책이 필요하다.

재연결을 개선할 때 exponential backoff만 보지 말고 catalog version과 in-flight call settlement를 함께 보자. stale tool invocation이 가장 위험한 경계다.

스터디에서 확인할 질문

server reconnect 후 같은 이름의 tool schema가 달라졌다면 현재 session에서 어떻게 version을 구분할 것인가?

plugin은 코드와 설정을 설치하는 공급망 경계다

검색·설치·업데이트·제거가 filesystem effect와 실행 가능 code admission을 포함한다.

PluginInstallationManager와 operations는 plugin artifact의 위치와 lifecycle을 다룬다. 설치 성공은 파일 복사만이 아니라 manifest 해석, dependency, 활성화 상태와 연결된다.

plugin이 MCP server나 hook을 포함하면 설치는 새로운 authority surface를 만든다. provenance, version pin, update policy, removal cleanup이 필요하다.

UI에서 "공식/설치됨" 배지는 trust를 암시하므로 실제 registry source와 검증 결과에 근거해야 한다. 이름이나 repository URL만으로 승격하면 안 된다.

스터디에서 확인할 질문

plugin 업데이트가 새로운 permission을 요구할 때 자동 업데이트를 계속할 것인가, 재승인을 요구할 것인가?

skill은 model 행동을 바꾸는 instruction dependency다

코드를 실행하지 않아도 prompt 우선순위와 tool 선택을 바꿀 수 있어 별도의 검토가 필요하다.

loadSkillsDir는 skill package를 발견하고 model context에 사용할 내용을 준비한다. bundled skill과 사용자/프로젝트 skill은 출처와 신뢰가 다르다.

skill을 단순 문서로 보면 prompt injection과 stale instruction 위험을 놓친다. 반대로 전부 code처럼 차단하면 사용자 확장성이 사라진다. scope, explicit invocation, allowed tools 같은 metadata가 중간 지대를 만든다.

skill 변경을 리뷰할 때 자연어 품질만 보지 말고 실행 범위, 충돌 규칙, context budget, 비밀 포함 여부를 확인하자.

스터디에서 확인할 질문

프로젝트가 제공한 skill이 사용자 global skill과 같은 이름일 때 어느 것을 선택하고 그 사실을 어떻게 표시해야 하는가?

04

Agent와 Task는 실행을 한 단계 더 중첩한다

subagent는 함수 호출이 아니라 독립 context·budget·permission·result projection을 가진 child runtime이며, task는 그 수명과 관찰을 관리한다.

이 장에서 실제로 펼쳐볼 경로

src/tools/AgentTool/** · src/tasks/** · src/Task.ts · src/tasks.ts

AgentTool은 새 conversation engine을 여는 도구다

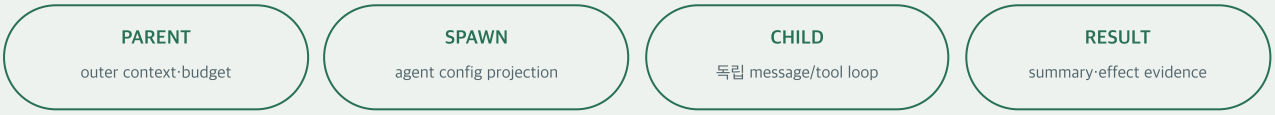
prompt 하나를 넘기는 것처럼 보여도 child의 도구, 모델, memory, permission을 다시 구성한다.

AgentTool과 runAgent 코드는 agent definition을 고르고 child context를 만들어 실행한다. parent는 child의 전체 transcript 대신 요약된 result와 progress를 받을 수 있다.

이 중첩은 context isolation과 병렬 탐색을 가능하게 하지만 비용과 authority가 증폭된다. child가 parent보다 넓은 tool set을 갖거나 permission prompt를 띄울 수 없으면 명시적 정책이 필요하다.

새 agent type을 추가할 때 persona보다 capability와 termination을 먼저 정의하자. 설명 문구만 다른 agent는 architecture상 별도 runtime일 이유가 약하다.

Parent와 child runtime의 경계



`tool_use` parent가 child request 생성

`progress` task projection만 parent에 전달

`complete` bounded result 합류

`cancel` child signal과 task status reconcile

언어·런타임 안쪽에서는 함수 호출 stack이 아니라 중첩된 state machine이다. child context isolation이 shared filesystem effect까지 격리하지는 않는다.

스터디에서 확인할 질문

child가 발견한 사실과 실제로 수행한 effect를 parent result에서 어떻게 구분해야 하는가?

Task는 background execution의 lifecycle record다

running, completed, failed, killed 상태가 UI pill보다 더 큰 의미를 가진다.

LocalAgentTask, LocalShellTask, RemoteAgentTask는 서로 다른 executor를 감싸지만 task id와 status, output 위치, stop 동작을 공통으로 제공한다. parent session은 이 상태를 attachment로 다시 context에 넣는다.

background task는 parent turn이 끝난 뒤에도 살아 있을 수 있다. 그래서 process-local object와 session durable reference를 구분해야 한다. 재개 후 running이라고 적힌 task가 실제로 존재하는지도 reconciliation이 필요하다.

task abstraction을 통합할 때 모든 종류를 동일 state machine으로 강제하지 말고 공통 lifecycle과 type-specific effect를 분리하자.

스터디에서 확인할 질문

process restart 뒤 이전 background task의 상태를 unknown으로 둘지 failed로 확정할지 무엇이 더 정직한가?

병렬 agent는 결과 수보다 context merge가 어렵다

여러 child가 같은 workspace를 읽고 쓰면 결과의 순서와 충돌을 parent가 해결해야 한다.

AgentTool이 여러 탐색을 동시에 수행하면 읽기 결과는 합치기 쉽지만 코드 변경은 shared-state collision을 만든다. task complete 순서가 논리적 우선순위는 아니다.

따라서 parallelism은 독립성 evidence가 있을 때만 이득이다. read-only 탐색, 별도 worktree, 명시적 ownership이 없으면 빠른 실행이 더 많은 reconciliation 비용을 만든다.

parent prompt에 결과를 단순 연결하면 상충 사실과 stale observation이 섞인다. source agent, revision, observed path를 보존한 merge가 필요하다.

스터디에서 확인할 질문

두 agent가 같은 파일에 다른 제안을 낸 경우 parent가 비교할 최소 evidence는 무엇인가?

영상의 UI와 generator를 이 축 안에서 읽는다

두 메커니즘은 제품의 전부가 아니라 runtime과 사람·host를 잇는 시간적 표현이다.

async generator 영상의 핵심은 중단 가능한 progressive event protocol을 이해하는 데 유효하다. React/Ink 영상의 핵심은 canonical state를 declarative terminal projection으로 만드는 데 유효하다.

그러나 generator만 보아서는 permission·durability를 설명할 수 없고, UI만 보아서는 effect semantics를 설명할 수 없다. 이 프로젝트에서 둘은 QueryEngine과 REPL/SDK adapter 사이를 이어 주는 메커니즘이다.

스터디 자료는 영상을 chapter 기준으로 승격하지 않고 실제 source path 안에 배치한다. 따라서 영상의 직관을 얻은 뒤 반드시 event identity, state ownership, failure path로 돌아온다.

스터디에서 확인할 질문

같은 partial message를 REPL과 SDK가 다르게 보여도 core 의미가 같다고 말하려면 무엇을 공유해야 하는가?

코드 뒤에서 작동하는 언어·런타임 메커니즘

문법 이름을 외우기보다, 런타임이 실제로 무엇을 저장하고 언제 제어를 넘기는지와 이 프로젝트가 그것을 어떤 책임에 사용하는지를 함께 본다.

React render/commit

언어가 제공하는 것
component를 순수 계산으로 호출한 뒤 변경분만 host renderer에 commit한다.

이 저장소가 쓰는 방식
Ink가 terminal 화면을 state의 projection으로 갱신한다.

놓치기 쉬운 함정
render 중 process/tool effect를 실행하면 재호출 때 중복될 수 있다.

context provider

언어가 제공하는 것
component subtree에 value를 전달하고 value identity 변화로 consumer render를 유발한다.

이 저장소가 쓰는 방식
modal, queue, notifications, overlay state를 공유한다.

놓치기 쉬운 함정
넓은 mutable value는 ownership과 rerender 범위를 과도하게 넓힌다.

async iterator protocol

언어가 제공하는 것
host가 next/return/throw를 호출해 producer와 양방향 종료 의미를 만든다.

이 저장소가 쓰는 방식
SDK caller가 engine event를 점진적으로 소비한다.

놓치기 쉬운 함정
network bridge의 queue와 durable resume는 iterator가 자동 제공하지 않는다.

runtime validation schema

언어가 제공하는 것
wire message를 parse해 control union을 좁힌다.

이 저장소가 쓰는 방식
SDK/remote의 permission·interrupt·configuration message를 검증한다.

놓치기 쉬운 함정
schema version과 late response correlation이 없으면 올바른 모양의 잘못된 응답이 된다.

adapter normalization

언어가 제공하는 것
외부 protocol의 이름·schema·result를 내부 contract로 변환한다.

이 저장소가 쓰는 방식
MCP tool과 resource를 Tool registry와 message system에 연결한다.

놓치기 쉬운 함정
연결 성공을 authority나 trust로 승격하면 안 된다.

child runtime isolation

언어가 제공하는 것
별도 context, budget, tool set, cancellation을 가진 중첩 실행을 만든다.

이 저장소가 쓰는 방식
AgentTool과 Task가 subagent/background work를 관리한다.

놓치기 쉬운 함정
workspace effect와 결과 merge까지 자동 격리되는 것은 아니다.

스터디용 코드 탐색 인덱스

파일을 처음부터 끝까지 읽지 않는다. 아래 시작점에서 symbol을 잡고 정의·호출·상태·실패 경로를 왕복한다.

시작 경로	먼저 볼 표현	그 다음에 물을 질문
src/screens/REPL.tsx	state/effect/provider wiring	canonical state와 ephemeral UI가 어디서 섞이는가
src/components/PromptInput/PromptInput.tsx	input mode/key handler	표시 문자열과 submit payload가 같은가
src/context	provider value와 command	projection이 domain state를 복제하는가
src/entrypoints/sdk/coreSchemas.ts	public event union	새 variant가 호환성에 어떤 영향을 주는가
src/entrypoints/sdk/controlSchemas.ts	request/response id	late response와 disconnect를 어떻게 처리하는가
src/services/mcp/normalization.ts	name/schema/result mapping	external server identity를 보존하는가
src/services/plugins/PluginInstallationManager.ts	install/update lifecycle	새 executable capability의 provenance는 무엇인가
src/tools/AgentTool/runAgent.ts	child config/result	parent보다 넓은 authority를 갖지 않는가

> **코드 독해 규칙**
>
> 타입 선언만 보고 runtime 검증을 가정하지 않는다. 호출 순서만 보고 authority를 가정하지 않는다. UI 표시만 보고 durable state를 가정하지 않는다. 주석만 보고 production 활성이나 upstream 의도를 확정하지 않는다.

같은 turn을 REPL과 SDK에서 비교하기

core event는 같고 projection과 control 방식만 달라지는지, 실제 코드의 타입과 state owner로 확인한다.

1. partial assistant message, permission request, tool progress, final result 네 사건을 고른다.
2. 각 사건이 QueryEngine에서 어떤 SDKMessage가 되고 REPL에서 어떤 state update와 component가 되는지 찾는다.
3. consumer가 느리거나 disconnect된 경우 event delivery와 engine 실행이 어떻게 달라지는지 적는다.
4. MCP tool 하나를 골라 discovery → registry → permission → result normalization을 추적한다.
5. AgentTool child task 하나를 parent session의 canonical state와 UI projection으로 나누어 그린다.

> **완료의 기준**
>
> UI, SDK, remote가 별도 제품 로직이 아니라 core protocol의 서로 다른 adapter/projection이며 각자 고유 실패를 가진다고 설명할 수 있으면 된다.

다음 모임에 남길 기록

다음 모임에는 projection 하나가 canonical state를 잘못 소유하고 있는 후보와 분리 방안을 가져온다.